Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

POO

José Miguel Paredes Sampaio
a106908

Gonçalo Francisco Loureiro Silva
a106811

José Simão Correia Campos
a107321

Data da Receção	
Responsável	
Avaliação	
Observações	

Programação Orientada a Objetos

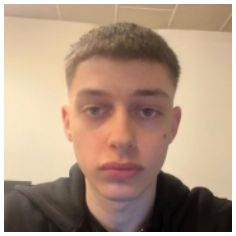
José Miguel Paredes Sampaio
a106908

Gonçalo Francisco Loureiro Silva
a106811

José Simão Correia Campos
a107321

maio, 2025

box



Índice

1. **Introdução** 1
2. **Arquitetura do Projeto** 2
3. **Descrição da aplicação** 3
4. **Análise e Discussão de Resultados** 6
5. **Conclusão** 7

Lista de Figuras

Figura 1 2
Figura 2 2
Figura 3 2
Figura 4 Diagrama de Classes 2
Figura 5 Menu inicial 3
Figura 6 Campos para registo 3
Figura 7 Campos de login 3
Figura 8 Menu principal do plano *PremiumTop e PremiumBase* 3
Figura 9 Menu principal do plano *Free* 4
Figura 10 Conteúdo da Biblioteca 4
Figura 11 Dados do Utilizador 5

1. Introdução

Neste trabalho tivemos como objetivo criar uma aplicação que faça a gestão de musicas e listas de reprodução de músicas que podem ser ouvidas pelos utilizadores. A nossa aplicação chama-se SpotifUM e de uma maneira geral ela permite aos utilizadores, dependendo do plano que tiverem, ouvir musica, organiza-la em *playlists*, bem como pesquisa-la através de álbuns ou nas próprias *playlists*. Os planos podem ser *Free*, que apenas permite ao utilizador ouvir musica aleatoriamente, ou *Premium* que permite a criação de *playlists* ou uma biblioteca com álbuns existentes. Se tiver o plano *PremiumTop*, tem ainda acesso a *playlists* geradas pela aplicação com base nas preferências pessoais.

2. Arquitetura do Projeto

A Figura 1 representa a arquitetura do nosso projeto. Os três construtores padrão, gets e sets de cada atributo e os metodos equals, tostring e clone são implementados em todas as classes, por isso não estão representados no diagrama.

As principais decições que tomamos na definição desta arquitetura foram:

- Existe uma hieraquia de planos e é possível alterar o plano de utilizador a qualquer momento.
- Cada utilizador tem uma biblioteca com as suas playlists privadas e albuns privados.
- Nem todos os planos permitem ter biblioteca mas mantivemos a biblioteca no utilizador para tornar mais fácil a manutenção da biblioteca do utilizador ao mudar de plano.
- O SpotifUM contém os Utilizadores, Albuns, Playlists, Artistas e Músicas por composição para não permitir à camada de interface acesso direto aos seus dados
- Por outro lado, dentro do SpotifUM as relações são de agregação uma vez que existem diversas partilhas de informação entre os objetos.
- Os dois pontos anteriores levaram à necessidade de usar métodos de cópia shallow e deep (ex: classe Album) conforme se criavam cópias para uso interno ou passar para a camada de interface

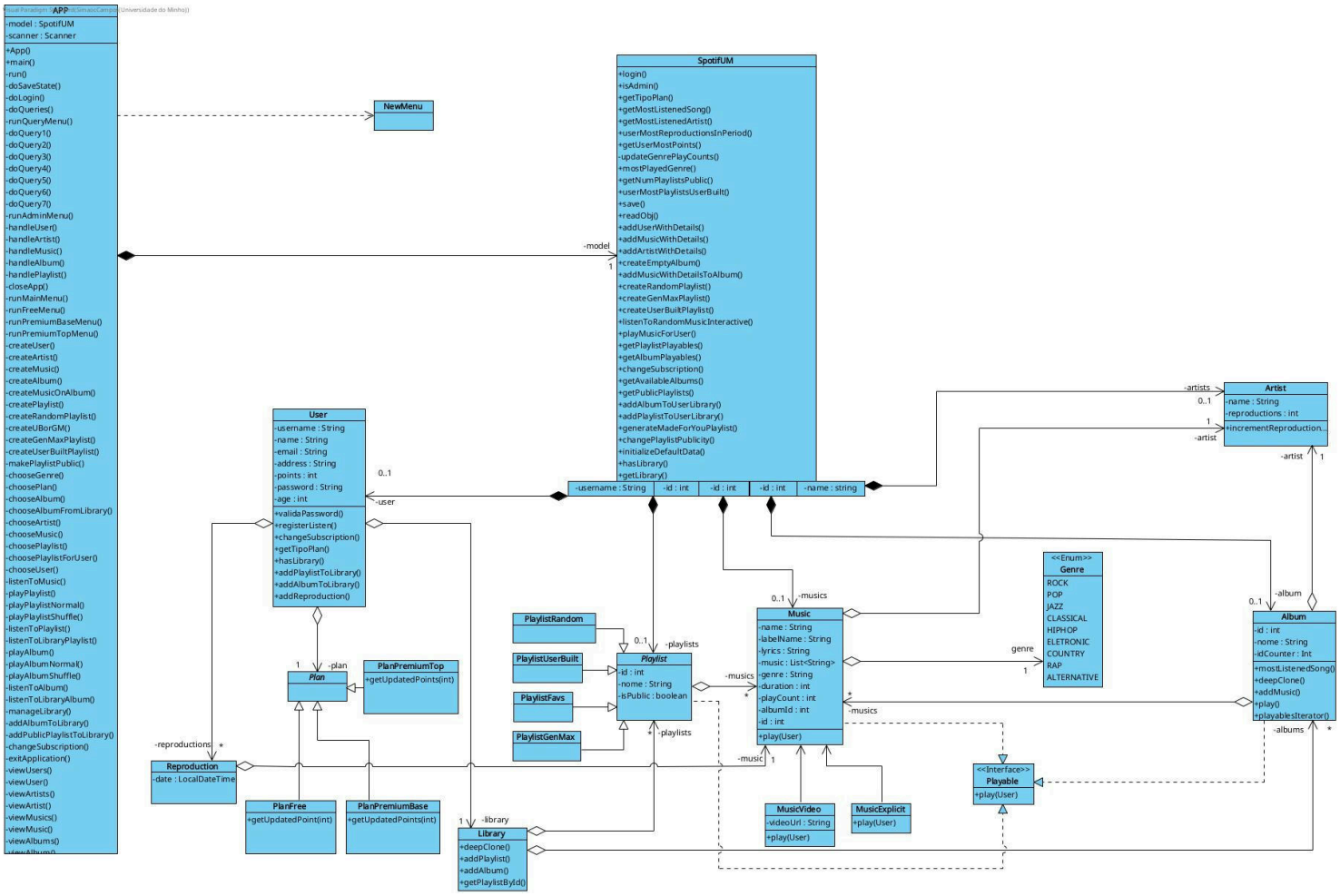


Figura 4: Diagrama de Classes

3. Descrição da aplicação

Inicialmente ao abrir a aplicação o utilizador encontra um menu inicial em que é possivel fazer login se já estiver registado na mesma, ou então registar-se.

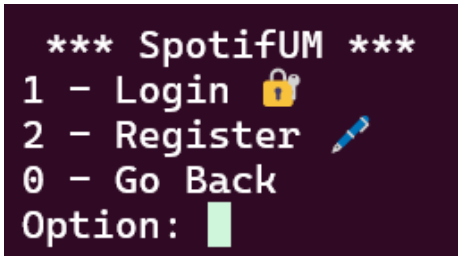


Figura 5: Menu inicial

Supondo que ainda não está registado, passamos a selecionar a opção 2 e preencher os campos pedidos.

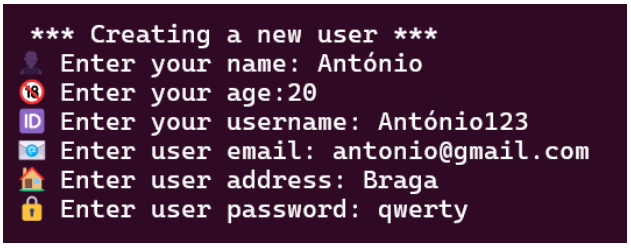


Figura 6: Campos para registo

Com o utilizador criado, podemos agora fazer login, selecionando a opção 1 e preenchendo os campos pedidos novamente.

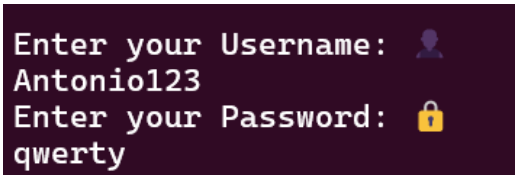


Figura 7: Campos de login

Depois de fazer login, dependendo do plano selecionado, o utilizador depara-se com um dos seguintes menus principais:

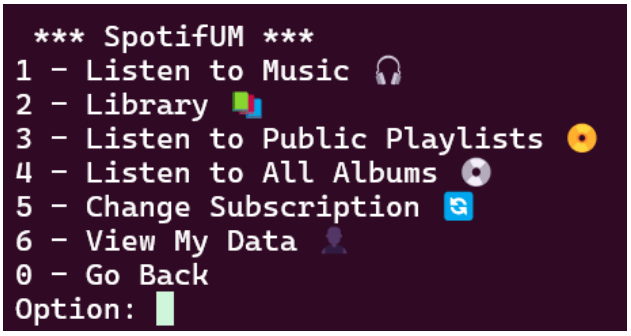


Figura 8: Menu principal do plano *PremiumTop e PremiumBase*

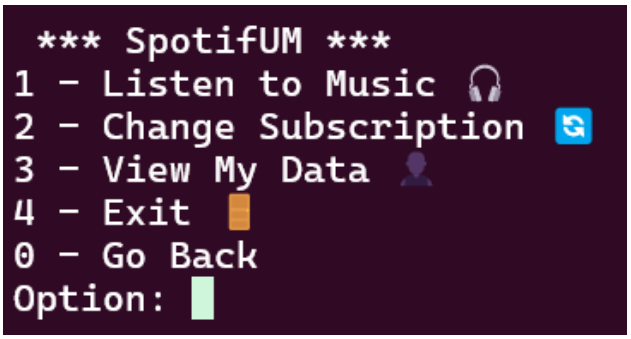


Figura 9: Menu principal do plano *Free*

Em ambos os menus, se for escolhida a opção 1, começa a tocar uma musica aleatoriamente, tendo a opção de avançar ou parar de tocar.

A opção 2 do menu *Free* ou opção 5 do menu *Premium* (Change Subscription) serve para o utilizador mudar de subscrição (plano) entre uma das 3 já mencionadas.

Nos menus dos utilizadores com planos Premium, a opção 2 acede à biblioteca (Figura 5), mostrando as seguintes opções:

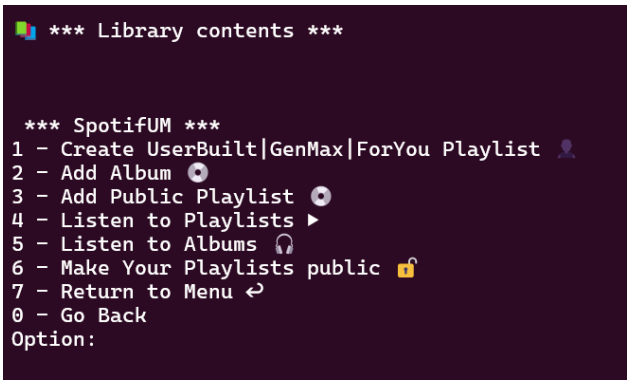


Figura 10: Conteúdo da Biblioteca

Neste caso a biblioteca deste utilizador está vazia, mas pode ser preenchida através das opções que podemos ver na imagem, como por exemplo criar uma playlist ou adicionar um álbum. É também possível ouvir as músicas que se encontram na biblioteca, bem como tornar as playlists publicas para os outros utilizadores. (Figura 7)

As opções 3 e 4 do menu dos planos *Premium* (Figura 5) servem respetivamente para ouvir as playlists publicas dos outros utilizadores e ouvir os álbuns já existentes.

A opção 3 do plano *Free* ou opção 6 dos planos *Premium* (Figuras 5 e 6) serve para ver os dados do utilizador que está a utilizar a aplicação.

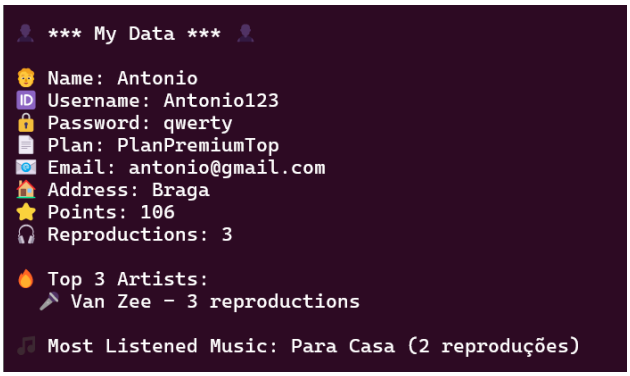


Figura 11: Dados do Utilizador

A opção 0 serve sempre para voltar atrás, sendo que se estiver no menu inicial fecha a aplicação.

4. Análise e Discussão de Resultados

Durante o desenvolvimento deste projeto, o nosso grupo conseguiu implementar praticamente todos os requisitos definidos no enunciado. As funcionalidades essenciais, como a criação de utilizadores, playlists, músicas e a reprodução de conteúdos, foram desenvolvidas com sucesso.

Adicionalmente, implementámos uma interface funcional, que permite aos utilizadores interagir com a aplicação. No entanto, por limitações de tempo, não tivemos oportunidade de a melhorar visualmente.

Alguns objetivos ficaram por concretizar, nomeadamente uma melhor divisão do projeto em módulos, de forma a garantir uma maior modularidade e organização interna. Também não tivemos tempo para implementar testes, o que seria importante para validar o funcionamento das várias componentes.

Em relação ao encapsulamento, procurámos respeitar este princípio ao longo de todo o projeto. Esforçámo-nos por garantir que as classes expunham apenas o necessário, evitando a partilha direta de estruturas de dados entre componentes. A principal dificuldade surgiu na definição clara dos limites de responsabilidade de cada classe, especialmente em momentos de maior interdependência entre os objeto0s. A gestão da composição e agregação simultaneamente foi algo que nos causou dificuldades ao longo do projeto. No entanto, achamos ter conseguido uma solução equilibrada

A arquitetura do projeto foi pensada de forma a permitir evoluções futuras. Por exemplo, a estrutura de playlists foi implementada com base em herança e polimorfismo, o que facilita a criação de novos tipos de playlists — bastando, para isso, criar novas classes especializadas que respeitem o contrato das existentes.

5. Conclusão

No geral, podemos considerar que o objetivo do trabalho foi bem concretizado. Todas as funcionalidades pedidas no relatório foram implementadas com sucesso.

Para além disso, este projeto revelou-se bastante útil para aprofundarmos os nossos conhecimentos em Java, não só ao nível da sintaxe da linguagem, mas também na aplicação de princípios da programação orientada aos objetos, como encapsulamento, herança e polimorfismo, num contexto prático e realista.